

Client-Side Security Review

Static Source Analysis of HTML & JavaScript Artifacts

Target artifacts	index.html, login.html, admin.html + bundled JavaScript
Origin	Offline mirror of hpacademy.com (captured with HTTrack 3.49-2)
Platform	SilverStripe CMS
Assessment type	White-box static analysis (client-side only)
Report date	02 June 2026
Classification	Confidential - for the system owner

This document reports security faults identified by reading the supplied HTML and JavaScript files. It explains, for each issue, how it could be exploited and how it should be remediated. Findings reflect only what is observable in the static client-side snapshot; see **Scope & Limitations** before acting on any item.

1. Scope & Methodology

The supplied folder is an offline copy of a live production website (High Performance Academy, hpacademy.com) mirrored with HTTrack. Three top-level HTML pages and their saved resources were provided. Analysis focused on the files containing first-party, site-authored code; well-known third-party scripts were identified and triaged rather than audited line-by-line.

Files reviewed

File	What it is	Reviewed
index.html	HTTrack mirror project-listing page (not the real homepage)	Yes
login.html	Member login page (SilverStripe + custom HPA login form)	Full
admin.html	CMS administration login page (/Security/login -> /admin/pages/)	Full
hpa-8529be3.js	Site's own webpack bundle (~5.6 MB): jQuery, jQuery UI, Bootstrap, TinyMCE, Moment, Algolia, site code	Full
10047999-10049486.js	Convert.com A/B-testing script (3rd-party; embeds visitor geo)	Triage
*.download (15 files)	GTM, Facebook Pixel, Hotjar, Matomo, Ortto, Reddit Pixel, identity/CDP cluster (identify/events/pixel/main)	Triage

Method

- Enumerated every HTML/JS file and separated first-party code from third-party scripts.
- Pattern-searched for dangerous DOM sinks (innerHTML, document.write, eval, location-based navigation), secrets/keys, inline event handlers, and form/credential handling.
- Fingerprinted bundled library versions and cross-referenced them against public CVE data.
- Traced selected source-to-sink data flows and inspected the surrounding code for context.

2. Scope & Limitations (read first)

- **Client-side only.** No server source code, database, or running instance was available. Server-side authentication, authorization and input validation could not be tested.
- **HTTP response headers are not captured** in an HTTrack mirror. Header-dependent controls (Content-Security-Policy, X-Frame-Options, HSTS, cookie Secure/HttpOnly/SameSite, cache-control) cannot be confirmed from these files. Findings that rely on headers are flagged as requiring server-side verification.
- **Static snapshot.** Values such as CSRF tokens and analytics IDs reflect one captured session; the live site may differ.
- **Third-party scripts** (GTM, Facebook, Hotjar, Matomo, Ortto, Reddit, Convert, the CDP cluster) were attributed and risk-rated by role, not audited internally.
- These are observations on a saved copy of a public, third-party production site. Remediation is the responsibility of that site's owner; this report is for educational/defensive use.

3. Executive Summary

The client-side code is built on a stack of **significantly outdated JavaScript libraries** with numerous published, exploitable vulnerabilities - the most serious finding. This is compounded by an **absence of integrity controls (SRI) and a heavy third-party script footprint, including session-recording, that loads directly on the login and CMS-admin pages**. The site's own anti-framing (clickjacking) defence has been deliberately commented out. The remaining items are lower-severity hardening and information-disclosure issues. No hard-coded administrative secret or private key was found in the client code. **Critically, these findings do not stand alone:** Section 5 shows how the uncontrolled third-party script footprint and the in-page CSRF token on the authentication pages combine into a credential- and session-theft chain whose combined business risk is CRITICAL - even though no single client-side finding is rated that high in isolation.

Severity	Count
HIGH	1
MEDIUM	3
LOW	3
INFO	2

Findings at a glance

ID	Finding	Severity
F-01	Outdated JavaScript libraries with known, exploitable CVEs	HIGH
F-02	No Subresource Integrity (SRI) on third-party scripts	MEDIUM
F-03	Tracking & session-recording loaded on login / admin pages	MEDIUM
F-04	Clickjacking protection disabled (frame-busting commented out)	MEDIUM
F-05	Reverse tabnabbing (target=_blank without rel=noopener)	LOW
F-06	Information disclosure (framework, tooling, service IDs)	LOW
F-07	DOM-based open-redirect pattern (locale switcher)	LOW
F-08	Hard-coded Algolia search key (public by design; abuse risk)	INFO
F-09	Sensitive-form hardening gaps (autocomplete / CSRF token)	INFO

4. Detailed Findings

F-01 Outdated JavaScript libraries with known, exploitable CVEs

HIGH

Location / Evidence

hpa-8529be3.js:34697 TinyMCE majorVersion '4' / minorVersion '9.11' (releaseDate 2020-07-13) [4.x is END-OF-LIFE]

hpa-8529be3.js:36136 jQuery.htmlPrefilter(elem) used to build innerHTML -> the exact CVE-2020-11022/11023 code path (jQuery < 3.5)

hpa-8529be3.js:92143 '/*! jQuery UI - v1.11.1 - 2014-08-19 ...'

hpa-8529be3.js:98481 'Bootstrap v3.2.0' (every component VERSION = '3.2.0')

hpa-8529be3.js:64083 moment hooks.version = '2.29.1'

Description

The site bundle ships several libraries that are years out of date and carry publicly documented, weaponised vulnerabilities. **jQuery** is pre-3.5 (the vulnerable `htmlPrefilter` routine is present) and is paired with **jQuery UI 1.11.1** (2014); affected by CVE-2020-11022 & CVE-2020-11023 (DOM XSS via manipulated HTML), CVE-2019-11358 (prototype pollution via `$.extend`), CVE-2015-9251 (cross-domain AJAX XSS), and jQuery-UI CVE-2021-41182/41183/41184 and CVE-2016-7103 (XSS via widget options). **Bootstrap 3.2.0** is affected by CVE-2019-8331, CVE-2018-14040/14041/14042 and CVE-2016-10735 - XSS through `data-*` attributes in tooltip, popover, collapse, scrollspy and the data API. **TinyMCE 4.9.11** is end-of-life and has multiple XSS advisories (e.g. CVE-2020-17480 and link/media URL XSS) fixed only from 5.6.0 onward. **Moment.js 2.29.1** is affected by CVE-2022-31129 (ReDoS) and CVE-2022-24785 (path traversal in locale loading).

How it could be exploited

Wherever the application feeds attacker-influenced data through one of these libraries' known sinks - e.g. a URL parameter or stored field rendered with a vulnerable jQuery/Bootstrap path, untrusted content opened in the TinyMCE editor, or a crafted locale string parsed by Moment - the attacker can achieve cross-site scripting or prototype pollution. On the login and CMS-admin pages in scope, a successful XSS lets an attacker run script in an authenticated admin's browser, steal the session cookie or CSRF token, submit the login/admin forms on their behalf, and take over the CMS. Prototype pollution can additionally corrupt application logic and chain into XSS or denial-of-service.

How to fix it

- Upgrade jQuery to $\geq 3.5.x$ and jQuery UI to $\geq 1.13.x$ (or remove jQuery UI if unused).
- Upgrade Bootstrap to $\geq 3.4.1$ (preferably migrate to 5.x); audit all `data-*` driven widgets.
- Migrate off TinyMCE 4.x to a supported release (6.x, or 5.10 LTS); sanitise editor content server-side.
- Upgrade Moment to $\geq 2.29.4$, or migrate to a maintained library (Luxon, day.js).
- Introduce automated dependency/CVE scanning in CI (npm audit, Dependabot, retire.js) so regressions are caught.

F-02 No Subresource Integrity (SRI) on third-party scripts

MEDIUM

Location / Evidence

login.html / admin.html: 0 occurrences of `integrity=` and 0 `crossorigin=` on any `<script>`

External script origins: googletagmanager.com, cdn.matomo.cloud, connect.facebook.net,

cdn3l.ink (Ortto), www.redditstatic.com, plus the identity/CDP cluster

Description

None of the externally hosted scripts are loaded with a Subresource Integrity hash or `crossorigin` attribute. The browser will execute whatever those third-party endpoints return, with no verification that the content is what the

site intended.

How it could be exploited

If any of these vendors - or a CDN/domain they rely on - is compromised or hijacked, the attacker's JavaScript runs with full DOM access on every page that includes the script, including the login and CMS-admin pages. This is the classic Magecart / supply-chain attack: the malicious script can silently key-log the email and password fields, read cookies, and exfiltrate credentials to an attacker server. Because the scripts are trusted implicitly, the attack is invisible to users.

How to fix it

- Add integrity (SHA-384) + crossorigin='anonymous' to every static third-party <script> tag.
- For tag-manager-injected scripts where SRI is impractical, enforce a strict Content-Security-Policy (script-src allowlist + nonces) to constrain what can execute.
- Self-host security-critical libraries so their integrity is under your control.
- Periodically review and prune the third-party script inventory.

F-03 Tracking and session-recording scripts loaded on login / admin pages

MEDIUM

Location / Evidence

Loaded on BOTH login.html and admin.html: hotjar-1298728.js (Hotjar session capture), fbevents.js + pixel id 705212672884340, redditstatic pixel (t2_jj8bavkc), container_mwhfPt9i.js (Matomo), GTM (server-side first-party path /coh4/), cdn3l.ink/app.js (Ortto), 10047999-10049486.js (Convert A/B - embeds geo: country 'AU', city 'PERTH'), identify_*.js + events.js + pixel.js + main.<hash>.js (data-id C7P06QQ40CK6SQS6AC2G) - identity/CDP cluster

Description

A large set of analytics, advertising, A/B-testing, visitor-identification and **session-recording** scripts is included directly on the authentication and CMS-admin login pages - the most sensitive pages on the site. Session-recording/heatmap tools such as Hotjar capture DOM mutations and, unless explicitly configured to mask them, user input.

How it could be exploited

Each third party is an implicit trust relationship and an additional credential-capture vector on the login/admin pages (compounding F-02). If a session-recording or analytics vendor is misconfigured or compromised, typed credentials and other sensitive form data can be recorded and stored off-site. Even absent compromise, capturing keystrokes/field values on a login page risks leaking passwords and PII into third-party platforms, and the embedded visitor geo-location data is exposed client-side.

How to fix it

- Remove non-essential trackers from authentication and admin pages; load them only where needed.
- In Hotjar and any session-replay tool, suppress input capture and explicitly mask password and sensitive fields (data-hj-suppress / equivalent).
- Minimise identity/CDP scripts on auth flows; document the data flows (DPIA).
- Constrain all of the above with a strict CSP (see F-02).

F-04 Clickjacking protection disabled (frame-busting commented out)

MEDIUM

Location / Evidence

```
hpa-8529be3.js ~91174-91179:

// Stop the site from being accessible when in an iFrame on another domain.

// Create a redirect loop so the site is unusable. ... (2024-04-16 Commented it out so
// that Jono doesn't have problems when creating tests in VWO)

// if (window.self !== window.top) { window.location.replace('https://www.hpacademy.com'); }
```

Description

The site previously shipped JavaScript frame-busting to prevent being embedded in a cross-domain iframe (added after a scam site framed it). That protection has been deliberately commented out to accommodate the VWO A/B-testing tool. Because the captured mirror does not include HTTP response headers, the presence of a server-side X-Frame-Options or CSP frame-ancestors directive could not be confirmed and must be verified.

How it could be exploited

If no header-based anti-framing control is present, an attacker can embed the login or CMS-admin page in a transparent iframe overlaid on decoy content (clickjacking / UI-redressing) and trick an authenticated administrator into performing actions or disclosing credentials. The same gap also re-enables the iframe-based impersonation the original code was written to stop.

How to fix it

- Enforce server-side Content-Security-Policy: frame-ancestors 'self' on all pages (and/or X-Frame-Options: SAMEORIGIN), prioritising /Security and /admin.
- Prefer header-based controls over JS frame-busting (which is bypassable).
- If framing is genuinely required for A/B tooling, allowlist only the specific trusted origins.

F-05 Reverse tabnabbing - target=_blank without rel=noopener

LOW

Location / Evidence

```
login.html: 9x target="_blank" | admin.html: 9x target="_blank" | rel="noopener"/norereferrer: 0

e.g. login.html:1211 <a class="facebook" href="https://www.facebook.com/HPAcademy"
target="_blank">
```

Description

Links that open in a new tab do not set rel="noopener norereferrer". The newly opened page receives a reference to the originating window via window.opener.

How it could be exploited

A malicious or compromised destination page can use window.opener.location = 'https://phishing-clone/login' to silently navigate the original tab to a credential-phishing copy of the site while the user is looking at the new tab. Modern browsers default target=_blank to no-opener behaviour (since ~2021), so practical impact is limited to older browsers and embedded web-views - hence the low rating.

How to fix it

Add rel="noopener norereferrer" to every anchor that uses target="_blank".

F-06 Information disclosure that aids fingerprinting

LOW

Location / Evidence

```
login.html:29 <meta name="generator" content="SilverStripe - http://silverstripe.org">
index.html:8 <!-- Mirror and index made by HTTrack Website Copier/3.49-2 ... -->
Client-exposed IDs: GTM-K49776B, FB pixel 705212672884340, Reddit t2_jj8bavkc,
Matomo container_mwhfPt9i, Ortto token, server-side GTM path /coh4/, CDP data-id
C7P06QQ40CK6SQS6AC2G
```

Description

The pages disclose the CMS in use (SilverStripe) and the mirroring tool, and embed a range of service/account identifiers. None of these is individually severe, but together they speed up reconnaissance.

How it could be exploited

Knowing the exact framework lets an attacker target framework-specific exploits and default paths (e.g. /Security, /admin). Exposed service IDs can be abused to pollute the owner's analytics or, where an endpoint lacks server-side rate-limiting, to drive cost/abuse against the associated service.

How to fix it

- Remove the generator meta tag and avoid leaking framework/version details.
- Treat any client-exposed ID/key as public and enforce server-side rate-limiting and abuse controls.
- Do not ship mirror/tooling banners in production artifacts.

F-07 DOM-based open-redirect pattern in the locale switcher

LOW

Location / Evidence

```
hpa-8529be3.js:91166-91171
$hpaFluentDropdown.on('change', function () { var value = $(this).val();
window.location.href = value; // navigates to a value taken from the <select>
Also: hpa-8529be3.js:108090 / 108276 window.location.href = $this.attr('href') / url
```

Description

Navigation targets are assigned to `window.location.href` directly from a DOM value (the selected locale option, and anchor hrefs elsewhere) without validating that the target is an allowed same-origin path. In the current markup the option values are server-rendered, so exploitability is low; this is flagged as a defense-in-depth weakness.

How it could be exploited

If any of these values can be influenced by an attacker (e.g. via injected markup from another flaw, or a future change that sources the value from user input/URL), it becomes an open redirect that can send users to an external phishing page, or - if a `javascript:` URI were accepted - lead to script execution.

How to fix it

- Before assigning to `location.href`, validate the target against an allowlist of known relative paths / same-origin URLs and reject absolute or scheme-bearing values.
- For the locale switcher, map option keys to a server-defined URL table rather than navigating to the raw option value.

F-08 Hard-coded Algolia search key (public by design, but abuse-prone)

INFO

Location / Evidence

```
hpa-8529be3.js:109873 algoliaAppID = 'MWM0VGEMN0'
```

```
hpa-8529be3.js:109874 algoliaSearchAPIKey = 'a19fb64b755e511f7574d6b0a2f7a107'
```

Description

The Algolia application ID and a **search-only** API key are embedded in client code. Algolia search-only keys are explicitly designed to be shipped to the browser, so this is **not** a leaked administrative/write secret.

Note

Reported as informational because exposure is expected for this key type; included for completeness and to recommend hardening.

How it could be exploited

The risk is limited to abuse of search itself: if the key is not restricted, anyone can issue unlimited queries (driving up operation costs) or systematically scrape the entire search index. It does not permit writing to or deleting from the index.

How to fix it

- Confirm the key is search-only, never an Admin key; rotate immediately if an Admin key was ever exposed.
- Apply Algolia key restrictions: rate limits, allowed indices, HTTP referer restrictions, and restricted query parameters; consider Secured API Keys for per-user scoping.

F-09 Sensitive-form hardening gaps (autocomplete / CSRF token)

INFO

Location / Evidence

```
admin.html:637 <input type="password" ... autocomplete="off"> (good)
```

```
login.html:623 / 678 password inputs have NO autocomplete attribute (inconsistent)
```

```
All forms share SecurityID value 5807d9494c344cbb39c4482890acb6463bfaef40 in this snapshot
```

Description

Password-field autocomplete handling is inconsistent between the admin and member login forms. The SilverStripe anti-CSRF SecurityID token appears with the same value across all forms in the mirror.

How it could be exploited

Inconsistent autocomplete is minor. The identical SecurityID is expected here because a static mirror captures a single session's token; it is therefore **not** evidence of a fixed/predictable token. However, if server-side the token were not bound to the session and rotated, CSRF protection on the login/admin forms would be undermined - this requires server-side verification.

How to fix it

- Verify SecurityID is per-session and rotated server-side (cannot be confirmed from a static mirror).
- Apply consistent autocomplete policy to credential fields across all forms.
- Ensure auth pages are served with Cache-Control: no-store and cookies use Secure/HttpOnly/SameSite (header verification required).

5. Compound Attack Chains (how individual findings combine)

Each finding in Section 4 is rated in isolation. Real incidents rarely rely on a single bug - an attacker combines several. The two authentication pages (login.html and admin.html) are where these findings converge, for two structural reasons:

- **The CSRF token is in the page.** SilverStripe's anti-CSRF SecurityID ships as a hidden field, so any script executing on the page can read it (login.html:641 / admin.html:655).
- **Credentials are entered here, beside uncontrolled code.** The login form posts Email and Password to hpacademy.com (login.html:615), while 6+ independent third-party scripts run on the same page with full DOM access and no SRI or CSP constraint (F-02, F-03).

Consequently, any technique that places attacker script on these pages yields immediate theft of **both** the typed credentials and a valid CSRF token. The chains below reach that state - or a comparable compromise - by different routes.

Chain overview

Chain	Combines	Combined risk
A - Supply-chain credential / session theft	F-02 + F-03 + F-09 + F-06 (F-01 multiplier)	CRITICAL
B - Trusted-domain phishing	F-06 + F-07 + F-05	HIGH
C - Forced authenticated form submission	auto-submit gadget + F-04 + F-06/F-07	MEDIUM-HIGH
D - Application XSS as force-multiplier	F-01 + conditional .html() sinks	CONDITIONAL

Chain A - Supply-chain credential and session theft (Magecart-style)

CRITICAL

Combines

F-02 (no SRI) + F-03 (third-party scripts on the auth pages) + F-09 (CSRF token in the DOM) + F-06 (reconnaissance). F-01 widens the foothold. No application-level XSS is strictly required.

Preconditions

Any one of the third-party providers whose script runs on the login/admin page - or a CDN or domain it depends on - is compromised, hijacked, or ships a malicious update (the routine Magecart pattern), and no Content-Security-Policy restricts which script origins may execute.

Key evidence

login.html:65-1224 external JS origins: googletagmanager.com, cdn.matomo.cloud, cdn3l.link (Ortto),
connect.facebook.net, analytics.tiktok.com, www.redditstatic.com (GTM injects more)
login.html / admin.html: 0 integrity= attributes on any <script> (F-02)
login.html:641 / admin.html:655 <input type="hidden" name="SecurityID" value="5807d949...">
login.html:615 <form action="https://www.hpacademy.com/account/HPALoginForm/" method="post"> (Email, Password)

Attack steps

1. The attacker compromises one trusted third-party tag (for example an analytics or marketing vendor, or a script injected at runtime by Google Tag Manager).
2. The swapped JavaScript loads on login.html / admin.html with full DOM access; the browser cannot tell it is malicious because no integrity hash (SRI) and no CSP constrain it.
3. It hooks the Email and Password fields and the form submit to capture credentials - the same input that the session-recorders in F-03 already passively observe.
4. It reads the SecurityID CSRF token from the hidden field, plus any non-HttpOnly cookies.

5. It exfiltrates the data to an attacker server, and can optionally submit the login or admin form to ride the authenticated session.

Outcome

Mass theft of member - and potentially CMS-administrator - credentials together with a valid CSRF token, leading to account and CMS takeover. The ingredients are individually rated MEDIUM/LOW/INFO, but combined on the credential-entry page the business risk is CRITICAL.

Chain B - Trusted-domain phishing

HIGH

Combines

F-06 (reconnaissance) + F-07 (DOM open-redirect) + F-05 (reverse tabnabbing).

Preconditions

The attacker can send a link to a member; the victim either follows a link on the genuine domain (F-07) or uses a browser or embedded web-view where window.opener remains available (F-05).

Key evidence

```
hpa-8529be3.js:91166 locale <select> 'change' handler: window.location.href = $(this).val() (no allow-list)
```

```
login.html:784-788, 1211 target="_blank" links with no rel="noopener"
```

Attack steps

1. Reconnaissance (F-06) confirms the SilverStripe platform and the /account member login.
2. The attacker distributes a link on the real hpacademy.com domain that either rides the unvalidated locale redirect to an external clone, or is a target=_blank link whose destination silently rewrites the original tab (reverse tabnabbing) to a login clone.
3. Because the link or origin is the genuine brand domain, the victim trusts it and re-enters credentials on the pixel-perfect clone.

Outcome

Credential theft via phishing that borrows the trust of the legitimate domain. Rated HIGH, gated by the social-engineering step.

Chain C - Forced authenticated form submission (newly identified gadget)

MEDIUM-HIGH

Combines

A newly identified URL-parameter-driven auto-submit routine + F-04 (clickjacking) + F-06/F-07 for delivery.

Preconditions

The victim is an authenticated member and opens an attacker-crafted hpacademy.com URL (or a framed copy of the page).

Key evidence

```
hpa-8529be3.js:92015 if (getQueryVariable('autoSubmit')) { var $form = $('#'+FormID); ... $form.submit(); }
```

```
hpa-8529be3.js:92020-92027 OptionalCourses1 / OpenPaymentPlanOptions set from URL query parameters
```

```
The page's server-rendered SecurityID is included in the auto-submitted POST -> a valid CSRF token rides along
```

Attack steps

1. When the URL contains autoSubmit=1, the bundle auto-populates the Combo Purchase form from query parameters (FormID, OptionalCourses1, OpenPaymentPlanOptions) and submits it.

2. The attacker crafts such a URL and delivers it (using Chain B's redirect or an email); merely opening it makes the logged-in victim's browser submit the form with attacker-chosen options, carrying the page's valid SecurityID token.
3. Clickjacking (F-04) provides a click-driven variant in which the page is framed beneath decoy content.

Outcome

Forced state-changing actions on behalf of an authenticated user - a CSRF-by-design gadget - such as initiating enrolments or purchases with attacker-chosen parameters. The ultimate financial impact depends on server-side purchase confirmation, which must be verified in a live test.

Chain D - Application XSS as a force-multiplier (conditional)

CONDITIONAL

Combines

F-01 (pre-3.5 jQuery) + the application's own .html() sinks.

Preconditions

A reachable injection point feeds one of the application's HTML sinks.

Key evidence

hpa-8529be3.js:36136 jQuery.htmlPrefilter() - the pre-3.5 path that widens what executes via .html()

hpa-8529be3.js:140818 / 140821 \$messages.html(response.error) ; \$el.html(response.data.credits) (fed by EVC API)

hpa-8529be3.js:138715 getUrlParameter(...) result is wrapped in forceInt() - that URL source is neutralised

Attack steps

1. Where attacker-influenced data reaches a .html() sink, the pre-3.5 jQuery turns markup that would be inert in a patched version into executing script.
2. The reachable first-party sinks are fed by EVC API response fields, so they execute attacker script only if that backend reflects attacker-influenced data without encoding it.

Outcome

Where a sink is reachable, full client-side script execution on an authenticated page - the same end-state as Chain A. Rated CONDITIONAL pending server-side confirmation.

Honest assessment. A deliberate search did not find an unconditional, attacker-controlled URL or hash value flowing into an HTML sink: the obvious URL source is integer-coerced (line 138715) and selector paths are guarded by a leading '#'. F-01 is therefore best treated as a force-multiplier on Chains A and B rather than a confirmed standalone client-only exploit.

5.1 One control breaks several chains

Because the chains converge on two conditions - untrusted script executing on the authentication pages, and pages that can be framed or redirected - a small set of controls disrupts multiple chains at once:

- **Strict Content-Security-Policy** (script-src allow-list with nonces; frame-ancestors 'self'): breaks Chain A by blocking rogue script origins and inline exfiltration, removes Chain C's clickjacking variant, and blunts Chain D.
- **Subresource Integrity** on static third-party scripts: removes the static-script vector of Chain A.
- **rel="noopener noreferrer"** on every target=_blank link: removes Chain B's tabnabbing route.
- **A same-origin allow-list** before any assignment to window.location: removes Chain B's open-redirect route.
- **An allow-list plus explicit user confirmation** on the autoSubmit routine, and server-side CSRF re-validation on state-changing POSTs: removes Chain C.
- **Library upgrades** (F-01): shrink Chain D and remove the XSS multiplier.

These chains are constructed from static evidence. Items marked conditional - the EVC HTML sink, the HttpOnly status of session cookies, and the financial impact of the autoSubmit gadget - depend on server-side behaviour that a static mirror cannot reveal and should be confirmed in a live, authorised test.

Appendix A - Bundled library versions & representative CVEs

Library	Version found	Status	Representative CVEs
jQuery	pre-3.5 (1.11.x era)	Vulnerable	CVE-2020-11022, CVE-2020-11023, CVE-2019-11358, CVE-2015-9251
jQuery UI	1.11.1 (2014)	Vulnerable	CVE-2021-41182/41183/41184, CVE-2016-7103
Bootstrap	3.2.0 (2014)	Vulnerable	CVE-2019-8331, CVE-2018-14040/14041/14042, CVE-2016-10735
TinyMCE	4.9.11 (2020)	End-of-life	CVE-2020-17480 + 4.x XSS advisories (fixed 5.6.0+)
Moment.js	2.29.1	Vulnerable	CVE-2022-31129 (ReDoS), CVE-2022-24785 (path traversal)

CVE identifiers are representative of the issues affecting each version, drawn from public vulnerability databases. jQuery's core version string in the bundle is the unsubstituted build placeholder '@VERSION'; the version was inferred from the presence of the pre-3.5 htmlPrefilter routine and the paired jQuery UI 1.11.1. Confirm exact versions and full CVE applicability against your build before remediation.

Appendix B - Third-party script inventory (login / admin pages)

Script / file	Vendor & purpose
gtm.js, /coh4/	Google Tag Manager (incl. server-side first-party container)
fbevents.js (id 705212672884340)	Facebook/Meta Pixel - advertising/conversion tracking
hotjar-1298728.js	Hotjar - heatmaps & session recording
container_mwhfPt9i.js	Matomo Tag Manager - analytics
app.js (cdn3l.ink)	Ortto - marketing automation capture
pixel.js (redditstatic)	Reddit Pixel - advertising
10047999-10049486.js	Convert.com - A/B testing (embeds visitor geo)
identify.js, events.js, pixel.js, main.js, main.js	Identify.me CDP cluster (data-id C7P06QQ40CK6SQS6AC2G)

End of report. Prepared from static analysis of the supplied HTML/JavaScript artifacts. Items dependent on HTTP response headers or server-side behaviour are flagged for verification in a live environment.